🏗️ API Design & System Architecture Document

Project: Zoho Recruit ATS Integration Microservice
Date: March 17, 2026
Version: 1.0

1. System Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          EXTERNAL CLIENTS                                 │
│  (Postman / Frontend Web App / Mobile App / Internal Tools)               │
└──────────────────────────────────────────────────────────────────────────┘
                                │
                                │ HTTP/HTTPS Requests
                                ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                     API LAYER (PORT 3000 / AWS API GW)                    │
│                                                                           │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │                    Serverless Offline / API Gateway              │   │
│  │  - Routing GET/POST requests                                     │   │
│  │  - CORS header injection                                         │   │
│  │  - Request/Response transformation                               │   │
│  └──────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────┬────────────────────────────────────────┘
                               │
                               │ Route to Handler
                               ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                     HANDLER LAYER (handler.py)                            │
│                                                                           │
│  ┌──────────────────────┐  ┌──────────────────────┐  ┌──────────────┐   │
│  │  get_jobs()          │  │ create_candidate()   │  │  get_apps()│   │
│  │  - Parse params      │  │ - Validate input     │  │ - Validate │   │
│  │  - Call client       │  │ - Call client        │  │ - Call cli │   │
│  │  - Return response   │  │ - Return 201 + IDs   │  │ - Paginate │   │
│  └──────────┬───────────┘  └──────────┬───────────┘  └──────┬───────┘   │
└─────────────┼──────────────────────────┼────────────────────┼──────────┘
              │                          │                    │
              └──────────────────────────┴────────────────────┘
                                │
                                │ Call Methods
                                ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                    CLIENT LAYER (zoho_client.py)                          │
│                                                                           │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │  OAuth Token Management                                          │   │
│  │  - _get_access_token() → Refresh token if expired               │   │
│  │  - _headers() → Build auth headers with Bearer token            │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                                                           │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │  Public Methods                                                  │   │
```

```
│  │  - get_jobs(page, limit)                                    │  │
│  │  - create_candidate(payload)                                │  │
│  │  - get_applications(job_id, page, limit)                   │  │
│  └──────────────────────────────────────────────────────────────┘  │
│                                                                     │
│  ┌──────────────────────────────────────────────────────────────┐  │
│  │  Helper Methods                                              │  │
│  │  - _map_job_status() → "open" → "OPEN"                      │  │
│  │  - _map_application_status() → "applied" → "APPLIED"        │  │
│  │  - _extract_location() → Extract city from object           │  │
│  │  - _split_name() → "John Doe" → ("John", "Doe")             │  │
│  │  - _handle_zoho_error() → Parse Zoho errors                 │  │
│  └──────────────────────────────────────────────────────────────┘  │
└─────────────────────────────────────────────────────────────────────┘
                                    │
                                    │ HTTP REST (Token + Org ID)
                                    ▼
┌─────────────────────────────────────────────────────────────────────┐
│                      ZOHO RECRUIT v2 API                            │
│                   https://recruit.zoho.in/recruit/v2                │
│                                                                     │
│   /jobs              [GET]  - List job openings                     │
│   /candidates        [POST] - Create candidate profile              │
│   /applications      [POST] - Link candidate to job                 │
│   /applications      [GET]  - Get applications by filter            │
│   /oauth/v2/token    [POST] - Refresh access token                 │
└─────────────────────────────────────────────────────────────────────┘
```

2. API Endpoint Specifications

2.1 GET /jobs

Purpose

Retrieve all open job positions from Zoho Recruit.

Request

Method: `GET`

Path: `/dev/jobs`

Host: `http://localhost:3000` (local) or AWS API Gateway URL (prod)

Query Parameters:

Parameter	Type	Required	Default	Description
page	integer	No	1	Page number for pagination (starts at 1)
limit	integer	No	50	Records per page (max 100)

Example Request:

```
curl "http://localhost:3000/dev/jobs?page=1&limit=20"
```

Response

Status Code: `200 OK`

Response Schema:

```
{
  "data": [
    {
      "id": "string",
      "title": "string",
      "location": "string",
      "status": "OPEN | CLOSED | DRAFT",
      "external_url": "string (URL)"
    }
  ],
  "pagination": {
    "page": "integer",
    "limit": "integer",
    "count": "integer (current page count)",
    "has_next": "boolean"
  }
}
```

Example Response:

```
{
  "data": [
    {
      "id": "ZR_1_JOB",
      "title": "Senior Python Developer",
      "location": "San Francisco, CA",
      "status": "OPEN",
      "external_url": "https://mituniversity.zoho.in/recruit/jobs/ZR_1_JOB"
    },
    {
      "id": "ZR_2_JOB",
      "title": "Frontend Engineer",
      "location": "New York, NY",
      "status": "OPEN",
      "external_url": "https://mituniversity.zoho.in/recruit/jobs/ZR_2_JOB"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "count": 2,
    "has_next": false
  }
}
```

Error Responses

400 Bad Request - Invalid query parameter

```
{
  "error": "Invalid page number"
}
```

500 Internal Server Error - Missing credentials

```
{
  "error": "Missing Zoho credentials in environment variables"
}
```

503 Service Unavailable - Zoho API down

```
{
  "error": "ATS request failed",
  "status_code": 503,
  "details": {
    "message": "Connection timeout to Zoho API",
    "code": "TIMEOUT"
  }
}
```

Field Mapping

Zoho Field	API Field	Transformation
`id`	`id`	Direct (string)
`title`	`title`	Direct
`city` or `location`	`location`	Extract first location
`status`	`status`	Normalize to OPEN/CLOSED/DRAFT
`url`	`external_url`	Direct (URL)

Implementation Notes

- Pagination is optional; default returns page 1
- If `limit` > 100, capped at 100 for performance
- Results ordered by job creation date (newest first)
- Includes both published and draft jobs (status differentiates)

2.2 POST /candidates

Purpose

Create a new candidate profile and immediately apply them to a specific job.

Request

Method: `POST`
Path: `/dev/candidates`
Content-Type: `application/json`

Request Body Schema:

```
{
  "name": "string (required)",
  "email": "string (required, email format)",
  "phone": "string (optional)",
  "resume_url": "string (optional, HTTPS URL)",
  "job_id": "string (required, must exist in Zoho)"
}
```

Example Request:

```
curl -X POST "http://localhost:3000/dev/candidates" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Akshay Jadhav",
    "email": "akshay@example.com",
    "phone": "+919876543210",
    "resume_url": "https://example.com/resume.pdf",
    "job_id": "ZR_1_JOB"
  }'
```

Response

Status Code: `201 Created`

Response Schema:

```
{
  "id": "string (candidate_id)",
  "name": "string",
  "email": "string",
  "job_id": "string",
  "application_id": "string"
}
```

Example Response:

```
{
  "id": "candidate_xyz123",
  "name": "Akshay Jadhav",
  "email": "akshay@example.com",
  "job_id": "ZR_1_JOB",
  "application_id": "app_abc456"
}
```

Error Responses

400 Bad Request - Missing required fields

```
{
  "error": "Missing required fields: name, email, job_id"
}
```

400 Bad Request - Invalid JSON

```
{
  "error": "Invalid JSON body"
}
```

404 Not Found - Job doesn't exist

```
{
  "error": "Job ID not found in Zoho",
  "details": "Verify job_id: ZR_1_JOB"
}
```

409 Conflict - Candidate already applied to this job

```
{
  "error": "Candidate already applied to this job"
}
```

502 Bad Gateway - Candidate created but app creation failed

```
{
  "error": "Candidate created but failed to attach to job",
  "candidate_id": "candidate_xyz123",
  "application_error": {
    "status_code": 400,
    "message": "Invalid job ID"
  }
}
```

503 Service Unavailable - Network error

```
{
  "error": "Network error while creating candidate",
  "details": "Connection timeout"
}
```

Field Mapping

API Field	Zoho Field	Notes
`name`	`first_name + last_name`	Split on first space
`email`	`email`	Validated format
`phone`	`phone`	Optional
`resume_url`	(stored as URL)	Optional
`job_id`	`job_id`	Must exist

Implementation Flow

```
1. Validate required fields (name, email, job_id)
2. Split name: "Akshay Jadhav" → first_name="Akshay", last_name="Jadhav"
3. POST /candidates to Zoho → Get candidate_id
4. POST /applications to link candidate to job
5. Return combined response with both IDs
6. If step 3 fails: return 500
7. If step 4 fails: return 502 (partial success)
```

Implementation Notes

- Creates candidate and applies to job in **2 API calls**
- If candidate already exists with same email, updates profile
- Atomic operation: both succeed or both fail
- Resume URL is metadata only (no file upload in this version)

2.3 GET /applications

Purpose

Retrieve all applications for a specific job with candidate details.

Request

Method: `GET`

Path: `/dev/applications`

Query Parameters:

Parameter	Type	Required	Default	Description
job_id	string	**Yes**	-	Job ID to filter applications
page	integer	No	1	Page number for pagination
limit	integer	No	50	Records per page
status	string	No	-	Filter by status (APPLIED, SCREENING, REJECTED, HIRED)

Example Requests:

```
# Get all applications for a job
curl "http://localhost:3000/dev/applications?job_id=ZR_1_JOB"

# Get with pagination
curl "http://localhost:3000/dev/applications?job_id=ZR_1_JOB&page=2&limit=25"

# Filter by status
curl "http://localhost:3000/dev/applications?job_id=ZR_1_JOB&status=SCREENING"
```

Response

Status Code: `200 OK`

Response Schema:

```
{
  "data": [
    {
      "id": "string (application_id)",
      "candidate_name": "string",
      "email": "string",
      "status": "APPLIED | SCREENING | REJECTED | HIRED"
    }
  ],
  "pagination": {
    "page": "integer",
    "limit": "integer",
    "count": "integer (current page count)",
    "has_next": "boolean"
  }
}
```

Example Response:

```
{
  "data": [
    {
      "id": "app_abc123",
      "candidate_name": "Akshay Jadhav",
      "email": "akshay@example.com",
      "status": "APPLIED"
    },
    {
      "id": "app_def456",
      "candidate_name": "John Doe",
      "email": "john@example.com",
      "status": "SCREENING"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 50,
    "count": 2,
    "has_next": false
  }
}
```

Error Responses

400 Bad Request - Missing job_id

```
{
  "error": "job_id query parameter is required"
}
```

400 Bad Request - Invalid status filter

```
{
  "error": "Invalid status. Must be one of: APPLIED, SCREENING, REJECTED, HIRED"
}
```

404 Not Found - Job doesn't exist

```
{
  "error": "Job not found",
  "details": "job_id: ZR_1_JOB"
}
```

500 Internal Server Error - Auth failure

```
{
  "error": "Zoho token endpoint returned no access_token"
}
```

503 Service Unavailable - Network issue

```
{
  "error": "Network error while fetching applications",
  "details": "Connection timeout to Zoho API"
}
```

Field Mapping

Zoho Field	API Field	Transformation
`id`	`id`	Direct
`candidate.name`	`candidate_name`	Extract from nested object
`email`	`email`	Direct
`status` or `stage`	`status`	Normalize (see mapping table below)

Status Mapping

Zoho Status	API Status
Applied, New, Submitted	APPLIED
Screening, Review, Under Review, Interview	SCREENING
Rejected, Disqualified	REJECTED
Hired, Offer Accepted, Joined	HIRED

Implementation Notes

- `job_id` is mandatory filter
- Status filter is optional; if provided, filters results
- Empty result set returns 200 OK (not 404)
- Sorted by application date (newest first)

3. Authentication & Security

3.1 OAuth 2.0 Flow Diagram

```
┌─────────────────────────────────────────────────────────────────┐
│                    INITIAL SETUP (One-time)                     │
└─────────────────────────────────────────────────────────────────┘

1. Developer creates App in Zoho Developer Console
   ↓
2. Gets: client_id, client_secret
   ↓
3. Runs OAuth flow:
   Browser → https://accounts.zoho.in/oauth/v2/auth?
             client_id=...&scope=...&redirect_uri=...
   ↓
4. User approves → Gets authorization code
   ↓
5. Backend exchanges code for tokens:
   POST /oauth/v2/token with code, client_id, client_secret
   ↓
6. Response includes: access_token + refresh_token
   ↓
7. Store refresh_token in .env (valid for ~2 years)

┌─────────────────────────────────────────────────────────────────┐
│                    RUNTIME (Every Request)                      │
└─────────────────────────────────────────────────────────────────┘

1. Lambda handler receives HTTP request
   ↓
2. ZohoClient._get_access_token() called
   ↓
3. Check: is access_token expired?
   ├─ NO → Use cached token
   └─ YES → Refresh using refresh_token
      └─ POST /oauth/v2/token with refresh_token
      └─ Get new access_token (valid 1 hour)
   ↓
4. Build Authorization header: "Zoho-oauthtoken {access_token}"
   ↓
5. Make Zoho API call with header
   ↓
6. Process response and return to client
```

3.2 Environment Variables

```
# OAuth Credentials (from Zoho Developer Console)
ZOHO_CLIENT_ID=1000.MC5RXTOUYV3G9DB5SYGRSEO39E1Y7U
ZOHO_CLIENT_SECRET=6ac1f4bc10be5f00490835ec6f448d5fdac083984b
ZOHO_REFRESH_TOKEN=1000.c04d7a7d7bf62c1eb9e555c653f6bef0...

# API Endpoints
ZOHO_RECRUIT_BASE_URL=https://recruit.zoho.in/recruit/v2
ZOHO_ACCOUNTS_BASE_URL=https://accounts.zoho.in

# Organization Context
ZOHO_ORG_ID=60067448187
```

3.3 Request/Response Header Specification

Outgoing Request to Zoho

```
Authorization: Zoho-oauthtoken 1000.ad79d94d4ce8e637e22c4c3333e8a347...
Content-Type: application/json
X-ORG-ID: 60067448187
User-Agent: zoho-recruit-microservice/1.0
Timeout: 20s
```

Incoming Response from Lambda

```
HTTP/1.1 200 OK
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
X-Request-ID: req-12345678 (optional)

{
  "data": [...],
  "pagination": {...}
}
```

4. Data Models

4.1 Job Model

```
interface Job {
  id: string;                    // Unique job ID (e.g., "ZR_1_JOB")
  title: string;                 // Job title (e.g., "Senior Python Developer")
  location: string;              // Location (e.g., "San Francisco, CA")
  status: "OPEN" | "CLOSED" | "DRAFT";  // Job status
  external_url: string;          // Shareable job URL
}
```

4.2 Candidate Model

```
interface Candidate {
  id: string;                    // Unique candidate ID
  name: string;                  // Full name
  email: string;                 // Email address
  phone?: string;                // Optional phone
  resume_url?: string;           // Optional resume URL
  job_id: string;                // Job applied for
  application_id: string;        // Application ID in ATS
}
```

4.3 Application Model

```
interface Application {
  id: string;                    // Application ID
  candidate_name: string;        // Candidate's full name
  email: string;                 // Candidate's email
  status: "APPLIED" | "SCREENING" | "REJECTED" | "HIRED";
  job_id: string;                // Job ID
  applied_date: string;          // ISO 8601 date (if available)
}
```

4.4 Pagination Model

```
interface Pagination {
  page: number;                  // Current page (1-indexed)
  limit: number;                 // Records per page
  count: number;                 // Records in this page
  has_next: boolean;             // More pages available?
}
```

4.5 Error Model

```
interface ErrorResponse {
  error: string;                 // Human-readable message
  status_code?: number;          // HTTP status code
  details?: {
    code?: string;               // Error code from Zoho
    message?: string;            // Zoho error message
  };
}
```

5. Sequence Diagrams

5.1 GET /jobs Sequence

```
Client           Lambda            ZohoClient       Zoho API
  |                |                    |              |
  |─GET /jobs──────►|                    |              |
  |                |                    |              |
  |                |──_get_access_token─►              |
  |                |                    |              |
  |                |        (if expired)               |
  |                |                    |──POST /token──►
  |                |                    |◄──access_token──
  |                |◄───token returned──|              |
  |                |                    |              |
  |                |──GET /jobs──────────────────────►
  |                |                    |    {headers} |
  |                |                    |              |
  |                |◄──[{job}, {...}]──────────────────
  |                |                    |              |
  |                |──_map_jobs()───────►              |
  |                |◄─[normalized jobs]─              |
  |                |                    |              |
  |◄─200 + {data}──|                    |              |
```

5.2 POST /candidates Sequence

```
Client           Lambda            ZohoClient       Zoho API
  |                |                    |              |
  |─POST candidate─►|                    |              |
  |  {name,...}     |                    |              |
  |                 |──validate()────────►              |
  |                 |◄──OK──────────────|              |
  |                 |                    |              |
  |                 |──create_candidate()|              |
  |                 |                    |              |
  |                 |──_get_access_token─►              |
  |                 |◄──access_token─────              |
  |                 |                    |              |
  |                 |──POST /candidates─────────────────►
  |                 |  {first_name,email}              |
  |                 |◄──{candidate_id}──────────────────
  |                 |                    |              |
  |                 |──POST /applications───────────────►
  |                 |  {candidate_id,job_id}           |
  |                 |◄──{application_id}────────────────
  |                 |                    |              |
  |◄─201 + {both IDs}                    |              |
```

5.3 GET /applications Sequence

```
Client           Lambda            ZohoClient       Zoho API
  |                |                    |              |
  |─GET /apps──────►|                    |              |
  |  ?job_id=...    |                    |              |
  |                 |──validate()────────►              |
  |                 |  (check job_id)    |              |
  |                 |◄──OK──────────────|              |
  |                 |                    |              |
  |                 |──_get_access_token─►              |
  |                 |◄──access_token─────              |
  |                 |                    |              |
  |                 |──GET /applications────────────────►
  |                 |  ?job_id=...       |              |
  |                 |◄──[{app},...] ────────────────────
  |                 |                    |              |
  |                 |──_map_apps()───────►              |
  |                 |◄─[normalized]──────              |
  |                 |                    |              |
  |◄─200 + {data}───|                    |              |
```

6. Error Handling Matrix

HTTP Status Codes

Code	Scenario	Action
200 OK	GET successful	Return data + pagination
201 Created	POST successful	Return created resource with ID
400 Bad Request	Invalid input, missing fields	Validate input, return error msg
401 Unauthorized	Invalid auth token	Refresh token or return auth error
404 Not Found	Resource doesn't exist	Return helpful 404 message

Code	Scenario	Action
409 Conflict	Duplicate candidate application	Handle gracefully, suggest action
500 Internal Server Error	Unexpected error, missing env vars	Log error, return 500 with message
502 Bad Gateway	Partial success (e.g., created candidate but failed app)	Return status with partial details
503 Service Unavailable	Zoho API unreachable, timeout	Return 503 with retry suggestion

Error Response Examples

Validation Error (400)

```
{
  "error": "Missing required fields: name, email",
  "details": {
    "provided": ["phone", "job_id"],
    "missing": ["name", "email"]
  }
}
```

Not Found (404)

```
{
  "error": "Job ID not found",
  "details": {
    "job_id": "INVALID_JOB",
    "suggestion": "Verify job_id in Zoho dashboard"
  }
}
```

Service Unavailable (503)

```
{
  "error": "Network error while fetching jobs",
  "details": {
    "cause": "Connection timeout to Zoho API",
    "retry_after": 5
  }
}
```

7. API Rate Limiting & Pagination

7.1 Pagination Strategy

Default: 50 records per page

```
GET /jobs?page=1&limit=20

Response:
{
  "data": [...20 records...],
  "pagination": {
    "page": 1,
    "limit": 20,
    "count": 20,
    "has_next": true   ← Client knows to fetch page 2
  }
}
```

Page Calculation:

- Offset = (page - 1) × limit
- Limit queries to first 100 records to prevent abuse

7.2 Rate Limiting (Future Feature)

```
Header: X-RateLimit-Limit: 1000
Header: X-RateLimit-Remaining: 999
Header: X-RateLimit-Reset: 1710685200

429 Too Many Requests:
{
  "error": "Rate limit exceeded",
  "retry_after": 60
}
```

8. Request/Response Examples

Example 1: Get Jobs (Complete Flow)

Request:

```
curl -X GET "http://localhost:3000/dev/jobs?page=1&limit=5"
```

Response (200):

```
{
  "data": [
    {
      "id": "ZR_1_JOB",
      "title": "Senior Python Developer",
      "location": "San Francisco, CA",
      "status": "OPEN",
      "external_url": "https://mituniversity.zoho.in/recruit/jobs/ZR_1_JOB"
    },
    {
      "id": "ZR_2_JOB",
      "title": "React Frontend Engineer",
      "location": "New York, NY",
      "status": "OPEN",
      "external_url": "https://mituniversity.zoho.in/recruit/jobs/ZR_2_JOB"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 5,
    "count": 2,
    "has_next": false
  }
}
```

Example 2: Create Candidate (Complete Flow)

Request:

```
curl -X POST "http://localhost:3000/dev/candidates" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Akshay Jadhav",
    "email": "akshay@mituniversity.edu.in",
    "phone": "+919876543210",
    "resume_url": "https://example.com/resume.pdf",
    "job_id": "ZR_1_JOB"
  }'
```

Response (201):

```
{
  "id": "candidate_xyz123abc",
  "name": "Akshay Jadhav",
  "email": "akshay@mituniversity.edu.in",
  "job_id": "ZR_1_JOB",
  "application_id": "app_abc456def"
}
```

Response (400 - Validation):

```
{
  "error": "Missing required fields: name, email, job_id"
}
```

Example 3: Get Applications (Complete Flow)

Request:

```
curl -X GET "http://localhost:3000/dev/applications?job_id=ZR_1_JOB&page=1&status=SCREENING"
```

Response (200):

```
{
  "data": [
    {
      "id": "app_abc456def",
      "candidate_name": "Akshay Jadhav",
      "email": "akshay@mituniversity.edu.in",
      "status": "SCREENING"
    },
    {
      "id": "app_def789ghi",
      "candidate_name": "Jane Smith",
      "email": "jane@example.com",
      "status": "SCREENING"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 50,
    "count": 2,
    "has_next": false
  }
}
```

Response (400 - Missing Parameter):

```
{
  "error": "job_id query parameter is required"
}
```

9. API Documentation (OpenAPI 3.0)

9.1 OpenAPI Schema Preview

```yaml
openapi: 3.0.0
info:
  title: Zoho Recruit ATS Integration API
  version: 1.0.0
  description: Unified REST API for Zoho Recruit job, candidate, and application management

servers:
  - url: http://localhost:3000/dev
    description: Local development server
  - url: https://api.example.com/prod
    description: Production AWS Lambda endpoint

paths:
  /jobs:
    get:
      summary: List all open jobs
      operationId: getJobs
      parameters:
        - name: page
          in: query
          required: false
          schema:
            type: integer
            default: 1
        - name: limit
          in: query
          required: false
          schema:
            type: integer
            default: 50
      responses:
        '200':
          description: Successful response
          content:
            application/json:
              schema:
                type: object
                properties:
                  data:
                    type: array
                    items:
```

```yaml
                $ref: '#/components/schemas/Job'
            pagination:
              $ref: '#/components/schemas/Pagination'

  /candidates:
    post:
      summary: Create candidate and apply to job
      operationId: createCandidate
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CandidateInput'
      responses:
        '201':
          description: Candidate created successfully
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/CandidateResponse'
        '400':
          description: Validation error

  /applications:
    get:
      summary: List applications for a job
      operationId: getApplications
      parameters:
        - name: job_id
          in: query
          required: true
          schema:
            type: string
        - name: page
          in: query
          required: false
          schema:
            type: integer
            default: 1
        - name: limit
          in: query
          required: false
          schema:
            type: integer
            default: 50
```

```yaml
          - name: status
            in: query
            required: false
            schema:
              type: string
              enum: [APPLIED, SCREENING, REJECTED, HIRED]
        responses:
          '200':
            description: Successful response
            content:
              application/json:
                schema:
                  type: object
                  properties:
                    data:
                      type: array
                      items:
                        $ref: '#/components/schemas/Application'
                    pagination:
                      $ref: '#/components/schemas/Pagination'

  components:
    schemas:
      Job:
        type: object
        properties:
          id:
            type: string
          title:
            type: string
          location:
            type: string
          status:
            type: string
            enum: [OPEN, CLOSED, DRAFT]
          external_url:
            type: string

      CandidateInput:
        type: object
        required:
          - name
          - email
          - job_id
        properties:
          name:
```

```yaml
          type: string
        email:
          type: string
        phone:
          type: string
        resume_url:
          type: string
        job_id:
          type: string

    CandidateResponse:
      type: object
      properties:
        id:
          type: string
        name:
          type: string
        email:
          type: string
        job_id:
          type: string
        application_id:
          type: string

    Application:
      type: object
      properties:
        id:
          type: string
        candidate_name:
          type: string
        email:
          type: string
        status:
          type: string
          enum: [APPLIED, SCREENING, REJECTED, HIRED]

    Pagination:
      type: object
      properties:
        page:
          type: integer
        limit:
          type: integer
        count:
          type: integer
```

```
      has_next:
        type: boolean
```

10. API Testing Checklist

10.1 Functional Tests

- [ ] GET /jobs returns 200 with job list
- [ ] GET /jobs pagination works (page 2, limit 10)
- [ ] POST /candidates returns 201 with IDs
- [ ] POST /candidates validates required fields (400)
- [ ] POST /candidates rejects invalid job_id (404)
- [ ] GET /applications requires job_id parameter
- [ ] GET /applications returns 200 with applications
- [ ] GET /applications status filter works

10.2 Error Tests

- [ ] Missing token returns 500
- [ ] Expired token auto-refreshes
- [ ] Network timeout returns 503
- [ ] Invalid JSON body returns 400
- [ ] Invalid enum values rejected
- [ ] Duplicate applications handled gracefully

10.3 Performance Tests

- [ ] Pagination handles 1000+ records
- [ ] Response time < 2s for typical queries
- [ ] Concurrent requests don't block
- [ ] Token refresh doesn't leak memory

11. Deployment Architecture

Local (Serverless Offline)

```
Port 3000 → HTTP → handler.py → zoho_client.py → Zoho API
```

AWS Lambda

```
CloudFront (CDN)
    ↓
API Gateway (HTTPS endpoints)
    ↓
Lambda Function (handler.py)
    ↓
CloudWatch Logs (monitoring)
    ↓
Zoho API (HTTPS)
```

Environment Parity

Aspect	Local	AWS
Runtime	serverless-offline	AWS Lambda
Port	3000	API Gateway
Timeout	configurable	30s (default)
Concurrency	1	Unlimited
Cost	Free	Per request

Document Version: 1.0
Last Updated: March 17, 2026
Status: ✅ Complete